

EDA and data visualization

Monica Alexander

15/01/26

Table of contents

1 Overview	1
1.1 A note on packages	2
2 TTC subway delays	2
3 EDA and data viz	3
3.1 Data checks	4
3.1.1 Sanity Checks	4
3.1.2 Missing values	5
3.1.3 Duplicates?	6
3.2 Visualizing distributions	7
3.3 Visualizing time series	12
3.4 Visualizing relationships	14
3.5 PCA (additional)	15
4 Lab Exercises	18

1 Overview

This week we will be going through some exploratory data analysis (EDA) and data visualization steps in R. The aim is to get you used to some possible steps and tools that you could take to understand the main characteristics and potential issues in a dataset.

We will be using the [opendatatoronto](#) R package, which interfaces with the City of Toronto Open Data Portal.

A good resource is part 1 (especially chapters 3 and 7) of ‘R for Data Science’ by Hadley Wickham, available for free here: <https://r4ds.had.co.nz/>.

1.1 A note on packages

You may need to install various packages used (using the `install.packages` function). Load in all the packages we need:

```
library(opendatatoronto)
library(tidyverse)
library(stringr)
library(skimr) # EDA
library(visdat) # EDA
library(janitor)
library(lubridate)
library(ggrepel)
```

2 TTC subway delays

This package provides an interface to all data available on the [Open Data Portal](#) provided by the City of Toronto.

Use the `list_packages` function to look what's available

```
all_data <- list_packages(limit = 500)
head(all_data)
```

```
# A tibble: 6 x 11
  title          id    topics civic_issues publisher excerpt dataset_category
  <chr>          <chr> <chr>  <chr>         <chr>    <chr>    <chr>
1 Licensed Child C~ 059d~ Commu~ NULL        Children~ "Licen~ Map
2 Sewer Gravity Ma~ 71a2~ Water NULL        Toronto ~ "This ~ Map
3 Sewer Maintenanc~ 1cb1~ Water NULL        Toronto ~ "This ~ Map
4 Sewer Inlets      ff05~ Water NULL        Toronto ~ "This ~ Map
5 Watermains        4e09~ Water NULL        Toronto ~ "This ~ Map
6 Toronto Island F~ 000e~ Trans~ NULL        Parks, F~ "This ~ Table
# i 4 more variables: num_resources <int>, formats <chr>, refresh_rate <chr>,
#   last_refreshed <date>
```

Let's download the data on TTC subway delays in 2024.

```
res <- list_package_resources("996cfe8d-fb35-40ce-b569-698d51fc683b") # obtained code from s
res <- res |> mutate(year = str_extract(name, "202.?"))
delay_2024_ids <- res |> filter(year==2024) |> select(id) |> pull()

delay_2024 <- get_resource(delay_2024_ids)

# make the column names nicer to work with
delay_2024 <- clean_names(delay_2024)
```

Let's also download the delay code and readme, as reference.

```
# note: I obtained these codes from the 'id' column in the `res` object above
delay_codes <- get_resource("3900e649-f31e-4b79-9f20-4731bbfd94f7")
delay_data_codebook <- get_resource("ca43ac3d-3940-4315-889b-a9375e7b8aa4")
```

This dataset has a bunch of interesting variables. You can refer to the readme for descriptions. Our outcome of interest is `min_delay`, which give the delay in mins.

```
head(delay_2024)
```

```
# A tibble: 6 x 10
  date          time day  station code min_delay min_gap bound line
<dtm>          <chr> <chr> <chr>   <chr>   <dbl>   <dbl> <chr> <chr>
1 2024-01-01 00:00:00 02:00 Monday SHEPPARD~ MUI      0        0 N    YU
2 2024-01-01 00:00:00 02:00 Monday DUNDAS S~ MUIS      0        0 N    YU
3 2024-01-01 00:00:00 02:08 Monday DUNDAS S~ MUPAA     4       10 N    YU
4 2024-01-01 00:00:00 02:13 Monday KENNEDY ~ PUTDN    10       16 E    BD
5 2024-01-01 00:00:00 02:22 Monday BLOOR ST~ MUPAA     4       10 N    YU
6 2024-01-01 00:00:00 02:25 Monday ST CLAIR~ MUPAA     3        9 N    YU
# i 1 more variable: vehicle <dbl>
```

3 EDA and data viz

The following section highlights some tools that might be useful for you when you are getting used to a new dataset. There's no one way of exploration, but it's important to always keep in mind:

- what should your variables look like (type, values, distribution, etc)
- what would be surprising (outliers etc)

- what is your end goal (here, it might be understanding factors associated with delays, e.g. stations, time of year, time of day, etc)

In any data analysis project, if it turns out you have data issues, surprising values, missing data etc, it's important you **document** anything you found and the subsequent steps or **assumptions** you made before moving onto your data analysis / modeling.

3.1 Data checks

3.1.1 Sanity Checks

We need to check variables should be what they say they are. If they aren't, the natural next question is to what to do with issues (recode? remove?)

E.g. check days of week

```
unique(delay_2024$day)
```

```
[1] "Monday"    "Tuesday"   "Wednesday" "Thursday"  "Friday"    "Saturday"
[7] "Sunday"
```

Check lines: oh no. some issues here. Some have obvious recodes, others, not so much.

```
unique(delay_2024$line)
```

```
[1] "YU"          "BD"          "YUS"
[4] "YU/BD"       "SHP"         NA
[7] "BLOOR DANFORTH" "YU / BD"     "YU/ BD"
[10] "SRT"         "YUS/BD"      "SHEP"
[13] "LINE 1"      "TRACK LEVEL ACTIVITY" "YU & BD"
[16] "109 RANEE"   "ONGE-UNIVERSITY AND BL" "YU/BD/SHP"
[19] "BD/ YUS"     "BD/ YU"      "BD/YU"
[22] "BD / YU"     "20 CLIFFSIDE"
```

The `skimr` package might also be useful here

```
skim(delay_2024)
```

Table 1: Data summary

Name	delay_2024
Number of rows	26467
Number of columns	10
Column type frequency:	
character	6
numeric	3
POSIXct	1
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
time	0	1.00	5	5	0	1420	0
day	0	1.00	6	9	0	7	0
station	0	1.00	5	22	0	456	0
code	0	1.00	3	5	0	125	0
bound	9520	0.64	1	1	0	5	0
line	44	1.00	2	22	0	22	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
min_delay	0	1	2.81	10.09	0	0	0	4	716	
min_gap	0	1	4.11	10.86	0	0	0	8	719	
vehicle	0	1	3224.49	2747.75	0	0	5120	5606	9546	

Variable type: POSIXct

skim_variable	n_missing	complete_rate	min	max	median	n_unique
date	0	1	2024-01-01	2024-12-31	2024-07-02	366

3.1.2 Missing values

Calculate number of NAs by column

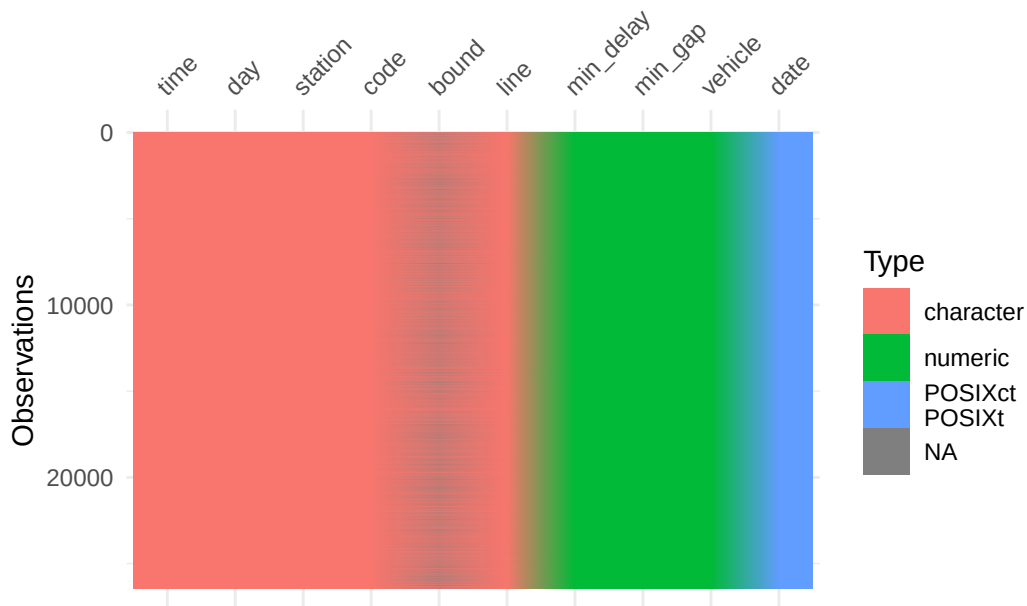
```
delay_2024 |>
  summarize(across(everything(), ~ sum(is.na(.x))))
```

```
# A tibble: 1 x 10
```

	date	time	day	station	code	min_delay	min_gap	bound	line	vehicle
	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>
1	0	0	0	0	0	0	0	9520	44	0

The `visdat` package is useful here, particularly to see how missing values are distributed. (commented out because couldn't get pdf to render in quarto)

```
vis_dat(delay_2024)
```



```
#vis_miss(delay_2024)
```

3.1.3 Duplicates?

The `get_dupes` function from the `janitor` package is useful for this.

```
get_dupes(delay_2024)
```

```
# A tibble: 32 x 11
  date           time day      station code min_delay min_gap bound line
  <dtm>          <chr> <chr>   <chr>   <chr>      <dbl>   <dbl> <chr> <chr>
1 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> SHP
2 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> SHP
3 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> SHP
4 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> YU/BD
5 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> YU/BD
6 2024-12-10 00:00:00 06:00 Tuesday TORONT~ MUO         0         0 <NA> YU/BD
7 2024-03-12 00:00:00 11:19 Tuesday EGLINT~ SUDP         3         6 S     YU
8 2024-03-12 00:00:00 11:19 Tuesday EGLINT~ SUDP         3         6 S     YU
9 2024-03-19 00:00:00 23:00 Tuesday LINE 2~ MUO         0         0 <NA> BD
10 2024-03-19 00:00:00 23:00 Tuesday LINE 2~ MUO         0         0 <NA> BD
# i 22 more rows
# i 2 more variables: vehicle <dbl>, dupe_count <int>
```

```
delay_2024 <- delay_2024 |> distinct()
```

3.2 Visualizing distributions

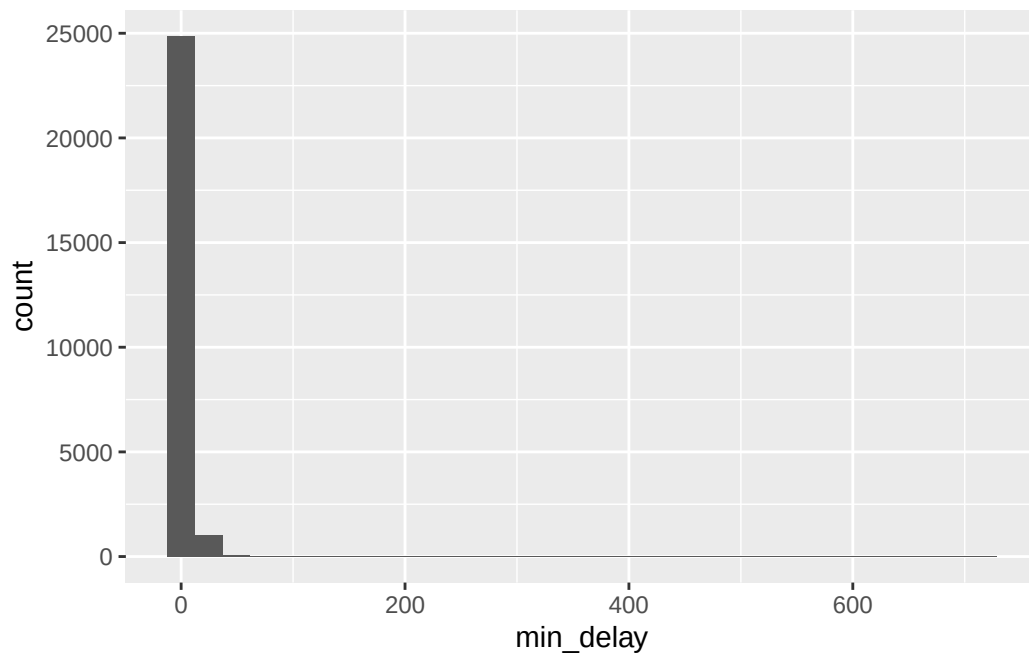
Histograms, barplots, and density plots are your friends here.

Let's look at the outcome of interest: `min_delay`. First of all just a histogram of all the data:

```
## Removing the observations that have non-standardized lines

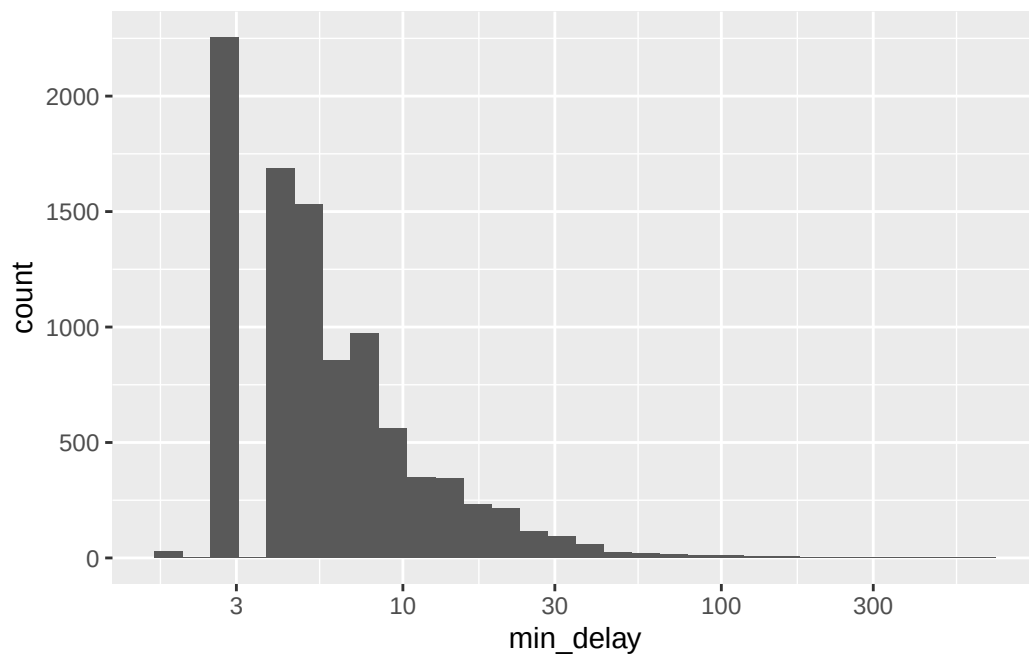
delay_2024 <- delay_2024 |> filter(line %in% c("BD", "YU", "SHP"))

ggplot(data = delay_2024) +
  geom_histogram(aes(x = min_delay))
```



To improve readability, could plot on logged scale:

```
ggplot(data = delay_2024) +  
  geom_histogram(aes(x = min_delay)) +  
  scale_x_log10()
```



Our initial EDA hinted at an outlying delay time, let's take a look at the largest delays below. Join the `delay_codes` dataset to see what the delay is. (Have to do some mangling as SRT has different codes).

```
delay_2024 <- delay_2024 |>
  left_join(delay_codes |> rename(code = `SUB RMENU CODE`, code_desc = `CODE DESCRIPTION...3

delay_2024 <- delay_2024 |>
  mutate(code_srt = ifelse(line=="SRT", code, "NA")) |>
  left_join(delay_codes |> rename(code_srt = `SRT RMENU CODE`, code_desc_srt = `CODE DESCRIP
  mutate(code = ifelse(code_srt=="NA", code, code_srt),
         code_desc = ifelse(is.na(code_desc_srt), code_desc, code_desc_srt)) |>
  select(-code_srt, -code_desc_srt)
```

The largest delay is due to Fires.

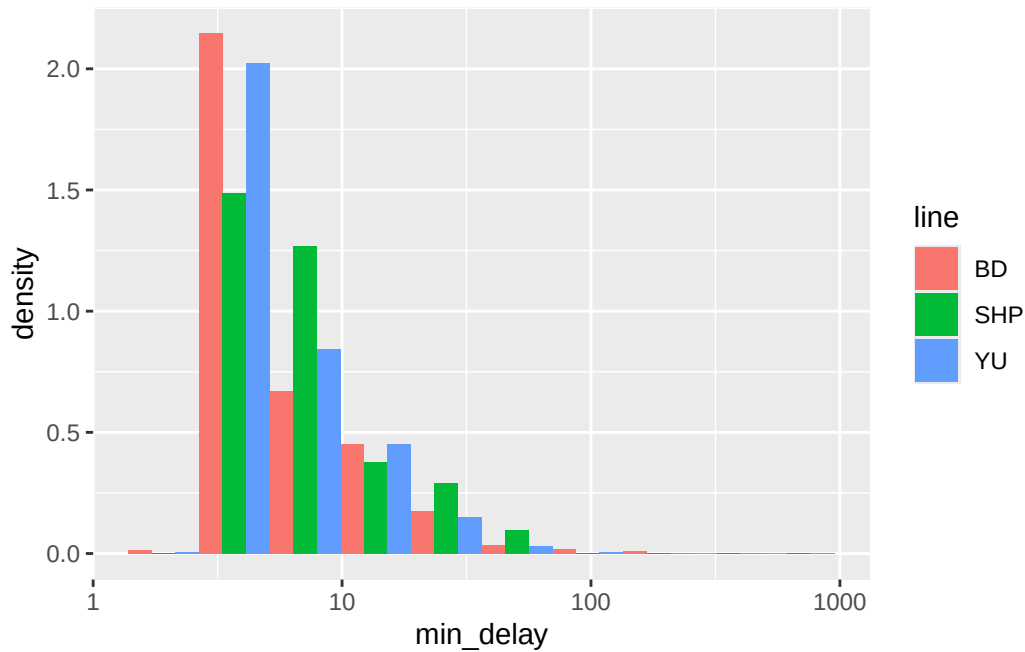
```
delay_2024 |>
  left_join(delay_codes |> rename(code = `SUB RMENU CODE`, code_desc = `CODE DESCRIPTION...3
  arrange(-min_delay) |>
  select(date, time, station, line, min_delay, code, code_desc)
```

```
# A tibble: 26,008 x 7
   date           time station          line min_delay code code_desc
<dtm>          <chr> <chr>          <chr>    <dbl> <chr> <chr>
1 2024-05-13 00:00:00 07:01 BROADVIEW AND ST G~ BD        716 EUCD Consequ~
2 2024-04-25 00:00:00 15:36 ISLINGTON TO KIPLI~ BD        640 MUPLB Fire/Smo~
3 2024-03-01 00:00:00 06:08 MUSEUM STATION      YU        491 PUTSM Track Sw~
4 2024-07-16 00:00:00 16:00 OLD MILL STATION BD        340 MUWEA Weather ~
5 2024-12-29 00:00:00 17:06 DAVISVILLE STATION YU        258 MUPLB Fire/Smo~
6 2024-10-23 00:00:00 07:42 HIGHWAY 407 STATION YU        257 MUPR1 Priority~
7 2024-04-26 00:00:00 12:06 SHERBOURNE STATION BD        217 SUPOL Held By ~
8 2024-07-16 00:00:00 12:41 KIPLING TO OLD MILL BD        197 PUSIS Signals ~
9 2024-07-07 00:00:00 07:59 KEELE STATION      BD        195 EUVE Work Veh~
10 2024-03-30 00:00:00 23:01 JANE STATION        BD        187 MUPR1 Priority~
# i 25,998 more rows
```

3.2.0.1 Grouping and small multiples

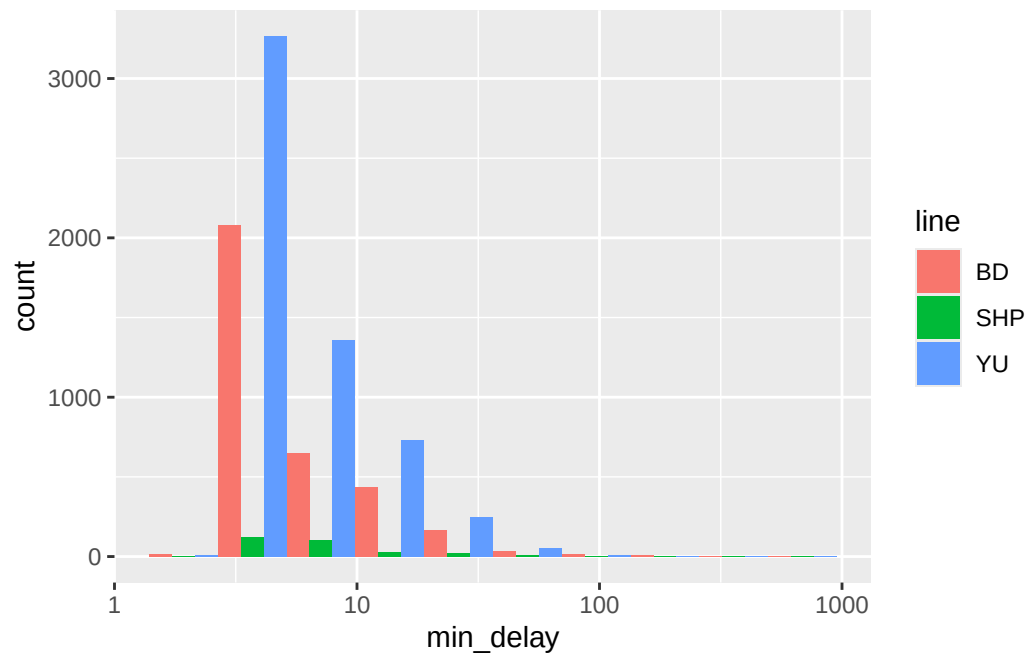
A quick and powerful visualization technique is to group the data by a variable of interest, e.g. `line`

```
ggplot(data = delay_2024) +
  geom_histogram(aes(x = min_delay, y = ..density.., fill = line), position = 'dodge', bins = 10) +
  scale_x_log10()
```



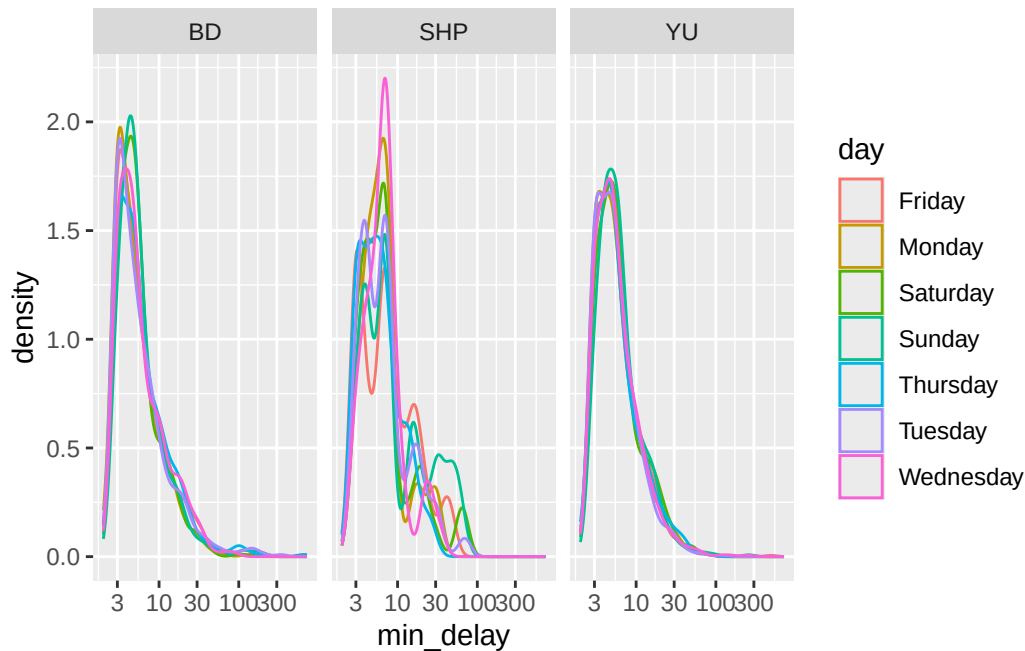
I switched to density above to look at the the distributions more comparably, but we should also be aware of differences in frequency, in particular, SHP and SRT have much smaller counts:

```
ggplot(data = delay_2024) +
  geom_histogram(aes(x = min_delay, fill = line), position = 'dodge', bins = 10) +
  scale_x_log10()
```



If you want to group by more than one variable, facets are good:

```
ggplot(data = delay_2024) +  
  geom_density(aes(x = min_delay, color = day), bw = .08) +  
  scale_x_log10() +  
  facet_wrap(~line)
```



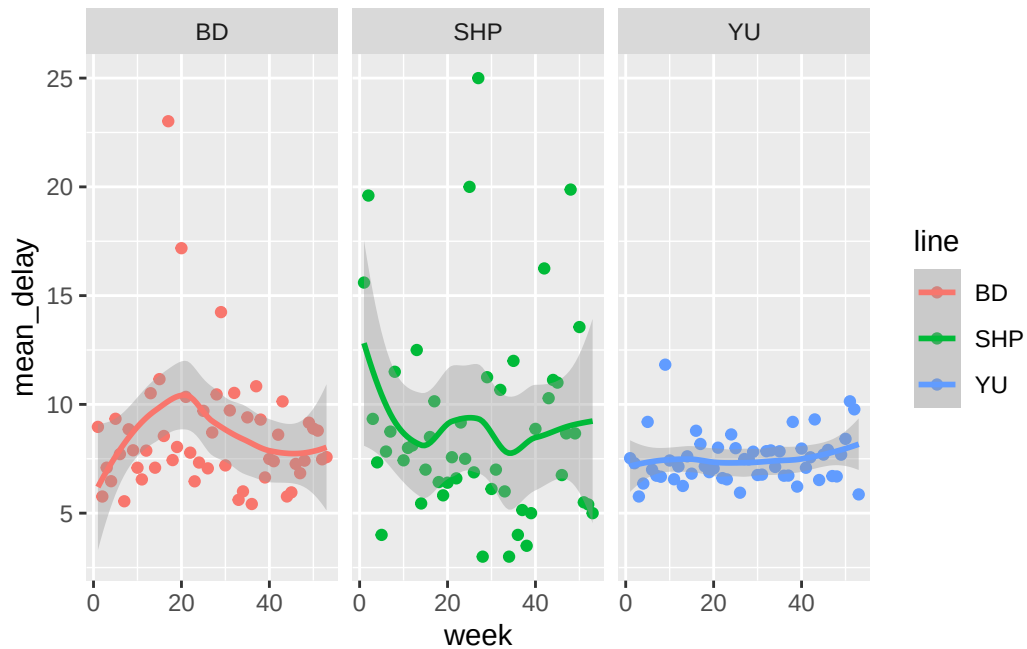
Side note: the station names are a mess. Try and clean up the station names a bit by taking just the first word (or, the first two if it starts with “ST”):

```
delay_2024 <- delay_2024 |>
  mutate(station_clean = ifelse(str_starts(station, "ST"), word(station, 1,2), word(station,
```

3.3 Visualizing time series

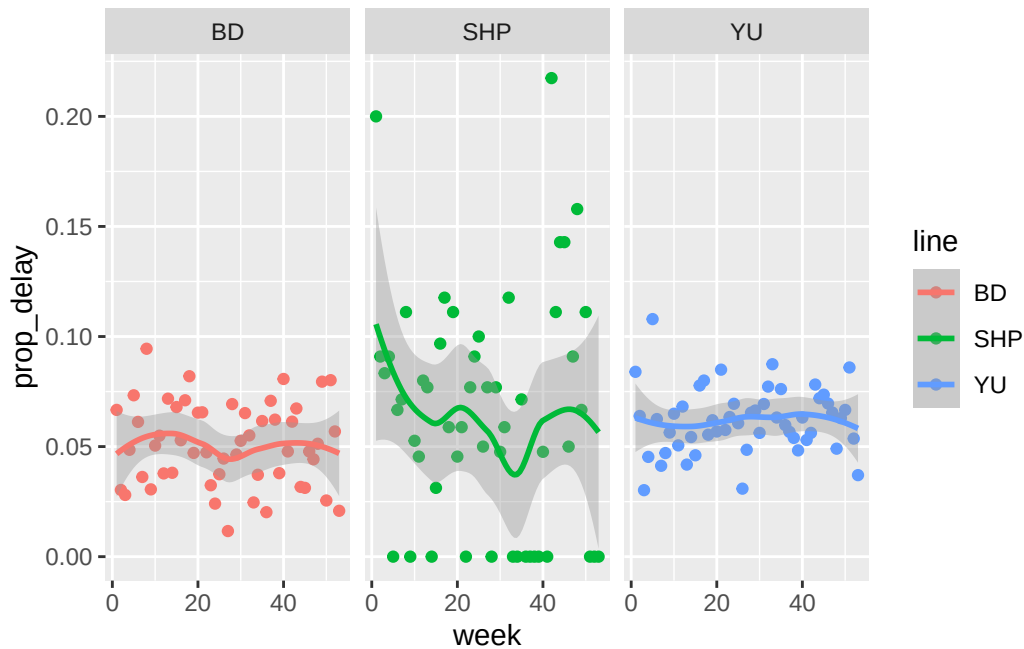
Daily plot is messy (you can check for yourself). Let’s look by week to see if there’s any seasonality. The `lubridate` package has lots of helpful functions that deal with date variables. First, mean delay (of those that were delayed more than 0 mins):

```
delay_2024 |>
  filter(min_delay>0) |>
  mutate(week = week(date)) |>
  group_by(week, line) |>
  summarise(mean_delay = mean(min_delay)) |>
  ggplot(aes(week, mean_delay, color = line)) +
  geom_point() +
  geom_smooth() +
  facet_grid(~line)
```



What about proportion of delays that were greater than 10 mins?

```
delay_2024 |>
  mutate(week = week(date)) |>
  group_by(week, line) |>
  summarise(prop_delay = sum(min_delay>10)/n()) |>
  ggplot(aes(week, prop_delay, color = line)) +
  geom_point() +
  geom_smooth() +
  facet_grid(~line)
```

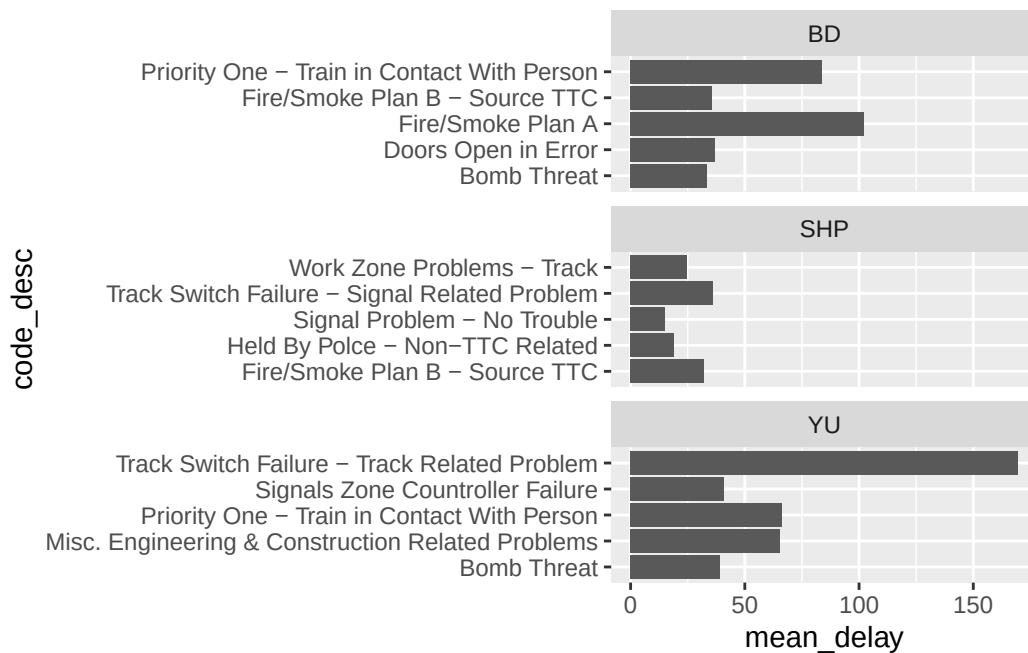


3.4 Visualizing relationships

Note that **scatter plots** are a good precursor to modeling, to visualize relationships between continuous variables. Nothing obvious to plot here, but easy to do with `geom_point`.

Look at top five reasons for delay by station. Do they differ? Think about how this could be modeled.

```
delay_2024 |>
  group_by(line, code_desc) |>
  summarise(mean_delay = mean(min_delay)) |>
  arrange(-mean_delay) |>
  slice(1:5) |>
  ggplot(aes(x = code_desc,
             y = mean_delay)) +
  geom_col() +
  facet_wrap(vars(line),
            scales = "free_y",
            nrow = 4) +
  coord_flip()
```



3.5 PCA (additional)

Principal components analysis is a really powerful exploratory tool, particularly when you have a lot of variables. It allows you to pick up potential clusters and/or outliers that can help to inform model building.

Let's do a quick (and imperfect) example looking at types of delays by station.

The delay categories are a bit of a mess, and there's hundreds of them. As a simple start, let's just take the first word:

```
delay_2024 <- delay_2024 |>
  mutate(code_red = case_when(
    str_starts(code_desc, "No") ~ word(code_desc, 1, 2),
    str_starts(code_desc, "Operator") ~ word(code_desc, 1,2),
    TRUE ~ word(code_desc,1))
  )
```

Let's also just restrict the analysis to causes that happen at least 50 times over 2024 To do the PCA, the dataframe also needs to be switched to wide format:

```

dwide <- delay_2024 |>
  group_by(line, station_clean) |>
  mutate(n_obs = n()) |>
  filter(n_obs>1) |>
  group_by(code_red) |>
  mutate(tot_delay = n()) |>
  arrange(tot_delay) |>
  filter(tot_delay>50) |>
  group_by(line, station_clean, code_red) |>
  summarise(n_delay = n()) |>
  pivot_wider(names_from = code_red, values_from = n_delay) |>
  mutate(
    across(everything(), ~ replace_na(.x, 0))
  )

```

Do the PCA:

```

delay_pca <- prcomp(dwide[,3:ncol(dwide)])

df_out <- as_tibble(delay_pca$x)
df_out <- bind_cols(dwide |> select(line, station_clean), df_out)
head(df_out)

```

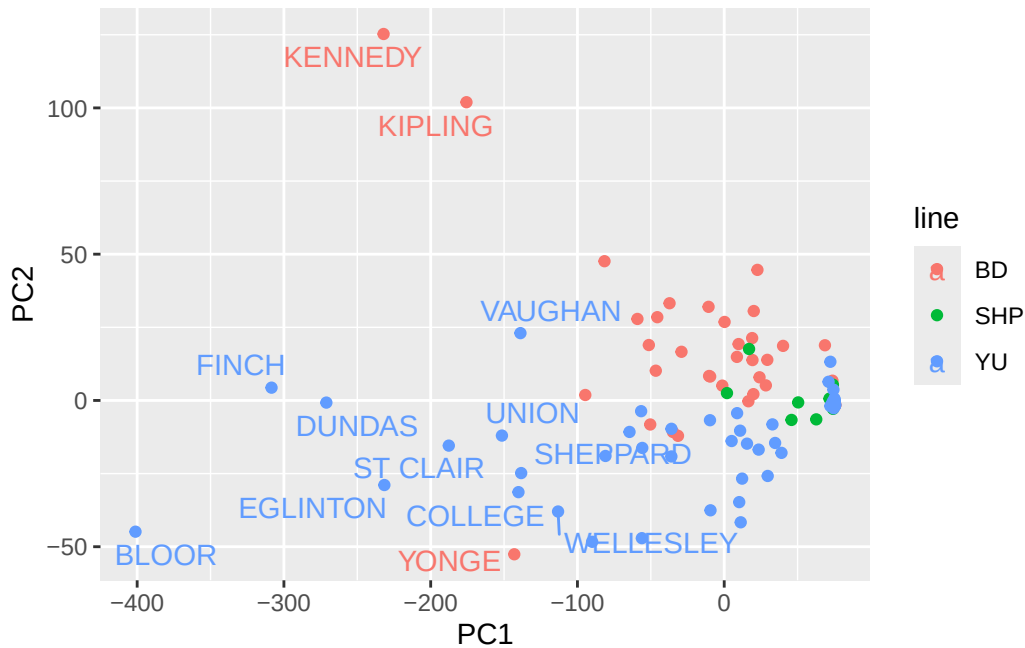
```

# A tibble: 6 x 42
# Groups:   line, station_clean [6]
  line station_clean  PC1  PC2  PC3  PC4  PC5  PC6  PC7  PC8
<chr> <chr>         <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BD   BATHURST     -51.4  18.9  4.72  22.3  6.53 -8.05  0.155  2.20
2 BD   BAY          -1.29  5.07  8.42  13.7  3.68 -0.870 -3.24  4.41
3 BD   BLOOR        68.6  18.9  5.93 -29.3 -13.4 -26.6  0.741 -1.40
4 BD   BROADVIEW    -59.1  27.9  3.40  23.4 -6.70 -11.3 -10.1 -7.78
5 BD   CASTLE       19.9  2.16  8.80  9.55 -4.30 -3.64 -1.48 -4.96
6 BD   CHANGE       75.5 -1.07  1.19  8.05 -3.97  0.0623 1.64  2.00
# i 32 more variables: PC9 <dbl>, PC10 <dbl>, PC11 <dbl>, PC12 <dbl>,
# PC13 <dbl>, PC14 <dbl>, PC15 <dbl>, PC16 <dbl>, PC17 <dbl>, PC18 <dbl>,
# PC19 <dbl>, PC20 <dbl>, PC21 <dbl>, PC22 <dbl>, PC23 <dbl>, PC24 <dbl>,
# PC25 <dbl>, PC26 <dbl>, PC27 <dbl>, PC28 <dbl>, PC29 <dbl>, PC30 <dbl>,
# PC31 <dbl>, PC32 <dbl>, PC33 <dbl>, PC34 <dbl>, PC35 <dbl>, PC36 <dbl>,
# PC37 <dbl>, PC38 <dbl>, PC39 <dbl>, PC40 <dbl>

```

Plot the first two PCs, and label some outlying stations:

```
ggplot(df_out,aes(x=PC1,y=PC2,color=line )) + geom_point() + geom_text_repel(data = df_out |
```



Plot the factor loadings. Some evidence of public v operator?

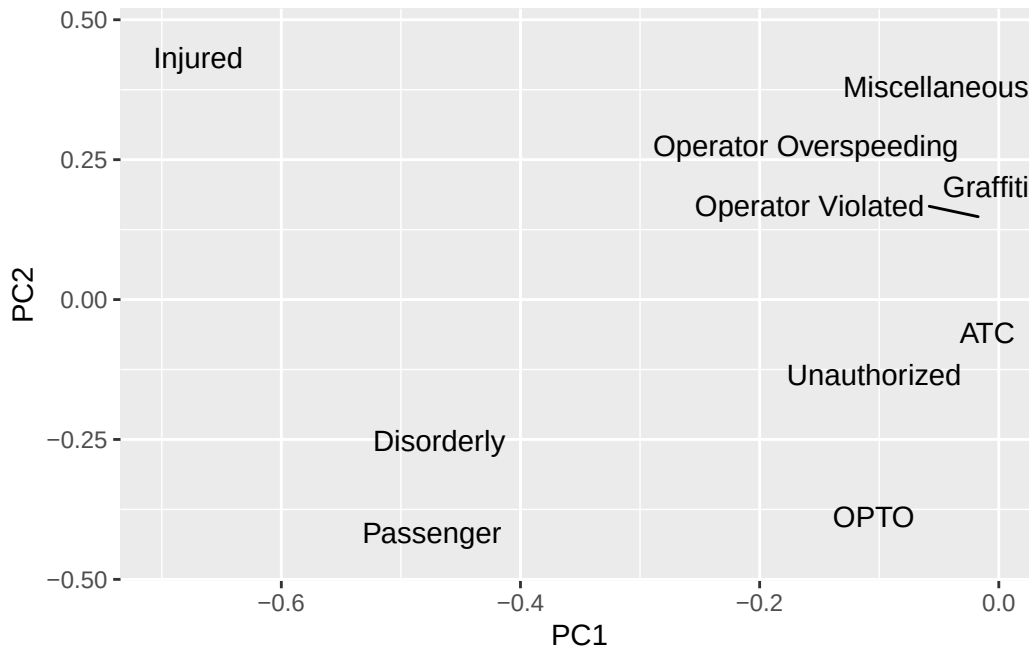
```
df_out_r <- as_tibble(delay_pca$rotation)
df_out_r$feature <- colnames(dwide[,3:ncol(dwide)])

df_out_r
```

```
# A tibble: 40 x 41
  PC1 PC2 PC3 PC4 PC5 PC6 PC7 PC8
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 -0.0914 -0.00737 0.0480 0.0134 0.0734 -0.0857 0.0245 -0.0587
2 -0.00494 0.00305 -0.00347 -0.0130 -0.00768 0.0111 0.00638 -0.0103
3 -0.00525 0.00184 -0.00285 -0.0111 0.00128 -0.00111 0.0171 0.00477
4 -0.438 -0.211 0.248 0.115 -0.458 -0.0217 -0.491 -0.278
5 -0.0434 0.0596 0.0276 -0.0694 0.0927 -0.0130 0.0981 -0.0753
6 -0.00830 0.0370 0.00798 -0.0271 0.0226 0.0535 0.0394 -0.0100
7 -0.0852 0.0326 -0.0419 -0.104 -0.0259 -0.0625 0.395 -0.0903
8 -0.00887 0.00205 0.0187 -0.0253 0.0277 0.00701 0.0434 -0.00893
9 -0.0306 0.163 -0.00877 -0.244 -0.116 0.354 -0.310 0.622
10 -0.00770 0.0108 0.00188 -0.0135 -0.00852 -0.0229 -0.00838 0.00164
# i 30 more rows
```

```
# i 33 more variables: PC9 <dbl>, PC10 <dbl>, PC11 <dbl>, PC12 <dbl>,
# PC13 <dbl>, PC14 <dbl>, PC15 <dbl>, PC16 <dbl>, PC17 <dbl>, PC18 <dbl>,
# PC19 <dbl>, PC20 <dbl>, PC21 <dbl>, PC22 <dbl>, PC23 <dbl>, PC24 <dbl>,
# PC25 <dbl>, PC26 <dbl>, PC27 <dbl>, PC28 <dbl>, PC29 <dbl>, PC30 <dbl>,
# PC31 <dbl>, PC32 <dbl>, PC33 <dbl>, PC34 <dbl>, PC35 <dbl>, PC36 <dbl>,
# PC37 <dbl>, PC38 <dbl>, PC39 <dbl>, PC40 <dbl>, feature <chr>
```

```
ggplot(df_out_r, aes(x=PC1, y=PC2, label=feature)) + geom_text_repel()
```



4 Lab Exercises

To be handed in via submission of quarto file (and rendered pdf) to GitHub.

1. Using the `delay_2024` data, plot the five stations with the highest mean delays. Facet the graph by `line`
2. Restrict the `delay_2024` to delays that are greater than 0 and to only have delay reasons that appear in the top 50% of most frequent delay reasons. Perform a regression to study the association between delay minutes, and two covariates: `line` and `delay reason`. It's up to you how to specify the model, but make sure it's appropriate to the data types. Comment briefly on the results, including whether results generally agree with the exploratory data analysis above.

3. Using the `opendatatoronto` package, download the data on mayoral campaign contributions for 2014 and clean it up. Hints:
 - find the ID code you need for the package you need by searching for ‘campaign’ in the `all_data` tibble above
 - you will then need to `list_package_resources` to get ID for the data file
 - note: the 2014 file you will get from `get_resource` has a bunch of different campaign contributions, so just keep the data that relates to the Mayor election
 - clean up the data format (fixing the parsing issue and standardizing the column names using `janitor`)
4. Summarize the variables in the dataset. Are there missing values, and if so, should we be worried about them? Is every variable in the format it should be? If not, create new variable(s) that are in the right format.
5. Visually explore the distribution of values of the contributions. What contributions are notable outliers? Do they share a similar characteristic(s)? It may be useful to plot the distribution of contributions without these outliers to get a better sense of the majority of the data.
6. List the top five candidates in each of these categories:
 - total contributions
 - mean contribution
 - number of contributions
7. Repeat 6 but without contributions from the candidates themselves.
8. How many contributors gave money to more than one candidate?